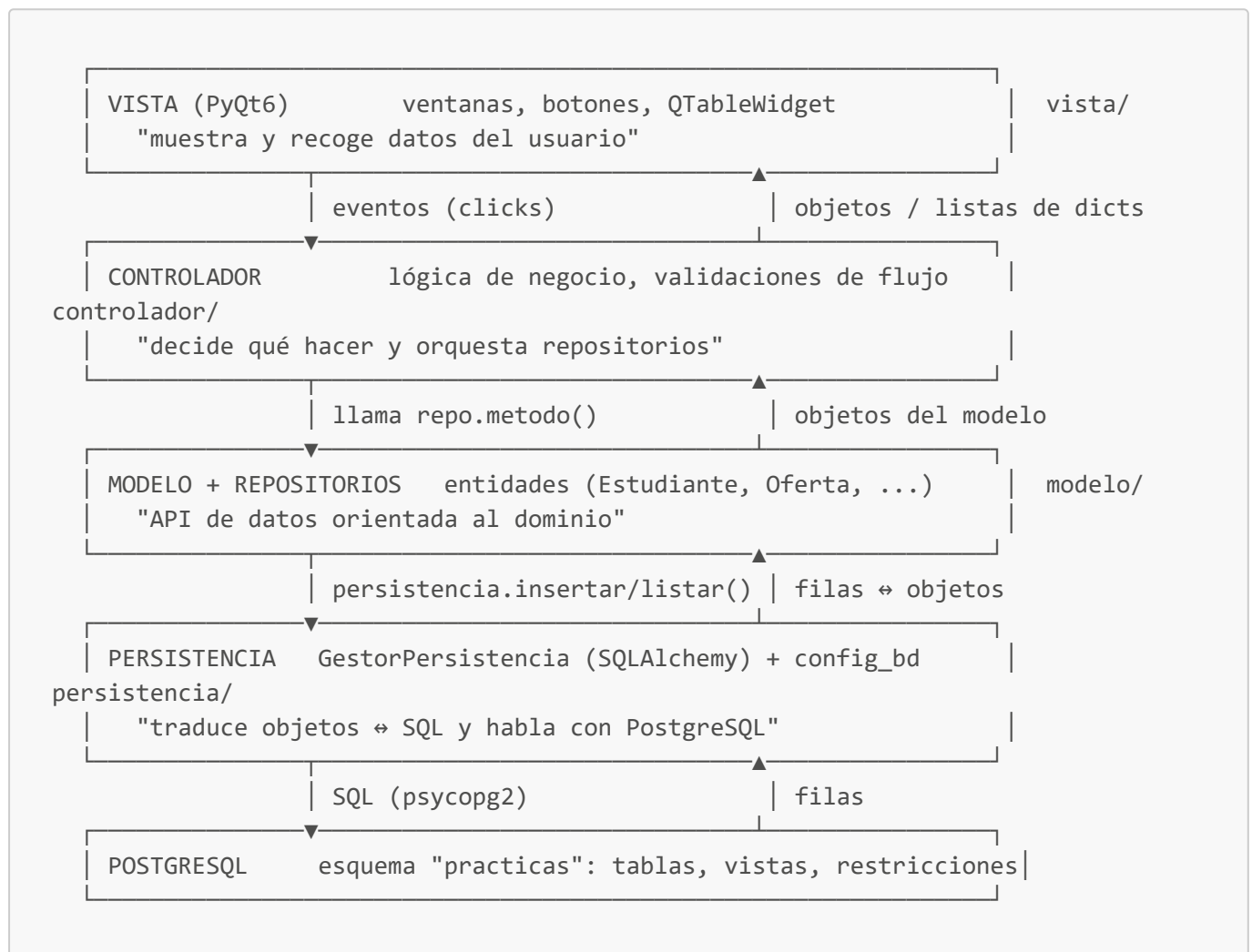


Funcionamiento de la capa PostgreSQL

Guía de exposición del Sistema de Prácticas Preprofesionales. Explica **cómo se define el esquema** (`esquema_postgresql.sql`), **cómo funciona `gestor_persistencia.py` con SQLAlchemy**, y **cómo el programa se conecta a la base de datos para consultar (y mostrar en tablas) o insertar datos**.

1. Visión general: arquitectura por capas

El proyecto separa responsabilidades en cuatro capas. Una acción del usuario viaja **de arriba hacia abajo** hasta la base de datos y los resultados **vuelven hacia arriba**:



Idea central: la vista y el controlador **nunca escriben SQL**. Solo hablan con los repositorios usando objetos (`Estudiante`, `Oferta`, ...). Todo el SQL está concentrado en una única clase: `GestorPersistencia`.

2. `esquema_postgresql.sql`: el contrato de la base de datos

2.1 ¿Qué es este archivo?

Es la **documentación de referencia** del esquema. Describe, en SQL puro y legible, exactamente las tablas, restricciones y vistas que la aplicación **crea automáticamente al arrancar** (de forma idempotente) desde

gestor_persistencia.py.

Importante para la defensa: el programa **no ejecuta este .sql**. Lo que hace es reconstruir ese mismo esquema mediante SQLAlchemy (sección 3). El **.sql** sirve para **leer y entender** el modelo, y para poder recrearlo a mano si hiciera falta.

2.2 Estructura del esquema

Todo vive bajo un **esquema** (namespace) llamado **practicas**:

```
CREATE SCHEMA IF NOT EXISTS practicas;
SET search_path TO practicas;
```

13 tablas organizadas en tres grupos:

Grupo	Tablas	Clave primaria
Usuarios	administrador, login, estudiante, tutor_academico, tutor_empresarial, coordinador_vinculacion	natural (usuario / cédula) → VARCHAR
Proceso	oferta, postulacion, practica, solicitud	subrogada (id generado) → INTEGER
Formularios	formulario1, formulario2, formulario3	subrogada

2.3 Decisiones de diseño (lo que hay que saber defender)

1. Claves naturales vs. subrogadas.

- Personas → su identificador real (cédula, usuario): **VARCHAR PRIMARY KEY**.
- Entidades del proceso (oferta, postulación, práctica, solicitud, formularios) → **id que genera la propia base** con **GENERATED ALWAYS AS IDENTITY**. La aplicación no lo inventa: lo **recupera** con **INSERT ... RETURNING**.

```
id_oferta INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

2. Integridad referencial con claves foráneas (FK). Por ejemplo, una postulación debe apuntar a un estudiante y a una oferta que existan:

```
CONSTRAINT fk_postulacion_cedula_estudiante
FOREIGN KEY (cedula_estudiante) REFERENCES practicas.estudiante(cedula)
```

3. Borrado lógico (no físico). Las FK **no** llevan **ON DELETE CASCADE**. En su lugar, cada tabla tiene una columna **eliminado** **BOOLEAN NOT NULL DEFAULT FALSE**. "Eliminar" = poner **eliminado = TRUE**. Así no se pierde historial y nunca se rompe una FK. La baja en cascada (eliminar un estudiante baja también sus postulaciones, prácticas...) la coordina el código: **controlador/eliminacion_cascada.py**.

4. Restricciones que reflejan reglas de negocio:

- **NOT NULL** (campos obligatorios), **UNIQUE** (email, *ruc_empresa*).
- **CHECK** para dominios cerrados y rangos:

```
CHECK (estado_validacion IN
('Pendiente', 'Validada', 'Enviada', 'Aceptada', 'Rechazada'))
CHECK (ciclo BETWEEN 1 AND 10)
CHECK (calificacion_sobre_100 BETWEEN 0 AND 100)
```

5. Tipos correctos (no todo es texto):

- Fechas → **DATE** (la app las muestra como dd/MM/yyyy).
- Dinero / notas → **NUMERIC(p,s)** (decimal exacto, nunca coma flotante).
- Estructuras anidadas (actividades, rúbricas, datos de empresa) → **JSONB**.

6. **Seguridad:** *contrasena* **VARCHAR(255)** guarda **siempre un hash con sal** (PBKDF2), nunca la contraseña en texto plano.

7. **Vistas** que concentran los **JOIN** de los listados dentro de la base: *vista_postulacion_detalle*, *vista_practica_detalle*, *vista_oferta_detalle*. Ejemplo (postulación + estudiante + oferta + empresa, todo en un solo SELECT):

```
CREATE OR REPLACE VIEW practicas.vista_postulacion_detalle AS
SELECT p.id_postulacion, p.estado_validacion, p.fecha, p.cedula_estudiante,
       e.nombres AS est_nombres, e.apellidos AS est_apellidos, e.ciclo AS
est_ciclo,
       o.puesto AS oferta_puesto, te.nombre_empresa
FROM practicas.postulacion p
JOIN practicas.estudiante e      ON p.cedula_estudiante = e.cedula
JOIN practicas.oferta o         ON p.id_oferta = o.id_oferta
JOIN practicas.tutor_empresarial te ON o.ruc_empresa = te.ruc_empresa;
```

2.4 Mapa de relaciones (FK)

```
tutor_empresarial(ruc_empresa) —< oferta —< postulacion >— estudiante
                                     |
coordinador_vinculacion ————— J (id_coordinador)
                                     |
                                     postulacion —< practica
                                     | (1:1 por
formulario)
                                     |
tutor_academico —┐┐ tutor_empresarial
                  |┐ practica —< formulario1 / 2 / 3
estudiante —< solicitud
```

3. gestor_persistencia.py: el motor con SQLAlchemy

3.1 ¿Qué librería usa y por qué?

Usa **SQLAlchemy 2.0**, el **ORM** (Object-Relational Mapper) estándar de Python. Un ORM traduce automáticamente entre **filas de la base** y **objetos de Python**, y genera el SQL por nosotros. Esto sustituye cientos de líneas de SQL escrito a mano por declaraciones de alto nivel, sin perder control ni seguridad.

Por debajo, SQLAlchemy sigue usando el driver **psycopg2** para hablar con PostgreSQL:

```
GestorPersistencia → SQLAlchemy (ORM) → psycopg2 (driver) → PostgreSQL
```

3.2 El esquema, declarado en Python (espejo del .sql)

Cada tabla del .sql se declara con objetos **Table/Column** de SQLAlchemy Core. Es la **misma información** (tipos, PK, FK, CHECK, UNIQUE, índices), pero en Python:

```
metadata = MetaData(schema="practicass")

tabla_postulacion = Table(
    "postulacion", metadata,
    Column("id_postulacion", Integer, Identity(always=True), primary_key=True),
    Column("fecha", FechaTexto),
    Column("estado_validacion", String(20), nullable=False),
    Column("cedula_estudiante", String(10),
           ForeignKey(tabla_estudiante.c.cedula), nullable=False, index=True),
    Column("id_oferta", Integer, ForeignKey(tabla_oferta.c.id_oferta),
           nullable=False, index=True),
    Column("id_coordinador", String(10),
           ForeignKey(tabla_coordinador_vinculacion.c.cedula)),
    Column("eliminado", Boolean, nullable=False, server_default=text("FALSE")),
    CheckConstraint(_in("estado_validacion", ESTADOS_POSTULACION),
                    name="ck_postulacion_estado"),
)
```

Esta tabla **Python** ⇔ tabla **SQL** es una correspondencia 1 a 1:

esquema_postgresql.sql	gestor_persistencia.py
VARCHAR(20)	String(20)
INTEGER GENERATED ALWAYS AS IDENTITY	Integer, Identity(always=True)
DATE	FechaTexto (tipo propio, ver 3.4)
NUMERIC(10,2)	Numeric(10, 2)
JSONB	JSONB
FOREIGN KEY (...) REFERENCES ...	ForeignKey(otra_tabla.c.columna)

esquema_postgresql.sql	gestor_persistencia.py
CHECK (...)	CheckConstraint("...")
CREATE INDEX	index=True / Index(...)

3.3 Mapeo imperativo: tablas ↔ clases del modelo

Las clases del dominio (*Estudiante*, *Oferta*, ...) viven en *modelo/* y **no saben nada de SQLAlchemy** (no lo importan ni heredan de ninguna base). El gestor las "conecta" con sus tablas mediante **mapeo imperativo**:

```
mapper_registry.map_imperatively(Estudiante, tabla_estudiante)
mapper_registry.map_imperatively(Postulacion, tabla_postulacion)
# ... una línea por entidad
mapper_registry.configure() # deja listos los descriptores de columna
```

A partir de aquí, cuando SQLAlchemy lee una fila de *postulacion*, **construye automáticamente un objeto *Postulacion*** con sus atributos rellenos — y lo hace **sin llamar al `__init__`** de la clase, para no re-validar datos que ya están guardados (es la técnica estándar de los ORM). Esto mantiene la separación de capas: el dominio queda limpio y la "magia" de persistencia vive solo aquí.

La entidad *solicitud* es la excepción: se maneja como **diccionario** (no tiene clase), usando SQLAlchemy Core directamente. Por eso *RepositorioSolicitud* accede con `sol["estado"]` en lugar de `sol.estado`.

3.4 Conversión de tipos automática (*TypeDecorator*)

Tres conversiones que antes se hacían a mano ahora son tipos personalizados que SQLAlchemy aplica solo, **al escribir y al leer**:

Tipo	En la base	En la aplicación	Qué hace
<i>FechaTexto</i>	DATE	texto "15/05/1980"	convierte <code>str</code> ↔ <code>date</code> en cada lectura/escritura
<i>ContrasenaHash</i>	VARCHAR(255)	texto plano de entrada	cifra la contraseña al guardar (si no está ya cifrada)
JSONB	jsonb	<code>dict</code> / <code>list</code> de Python	serializa/deserializa estructuras anidadas

```
class FechaTexto(TypeDecorator):
    impl = Date
    def process_bind_param(self, value, dialect): # Python -> BD (al escribir)
        if value in (None, ""): return None
        return datetime.strptime(value, "%d/%m/%Y").date() if isinstance(value,
str) else value
```

```
def process_result_value(self, value, dialect): # BD -> Python (al leer)
    return value.strftime("%d/%m/%Y") if value else value
```

3.5 La API pública (lo único que ven los repositorios)

GestorPersistencia expone un puñado de métodos. Los repositorios solo usan estos; no saben que por dentro hay SQLAlchemy:

Método	Qué hace	SQL equivalente
<code>obtener(entidad, clave)</code>	un objeto por su PK	<code>SELECT ... WHERE pk = ?</code>
<code>existe(entidad, clave)</code>	¿existe esa PK?	<code>SELECT 1 ... WHERE pk = ?</code>
<code>listar(entidad, where, params, ...)</code>	lista (excluye eliminados por defecto)	<code>SELECT ... WHERE eliminado=FALSE AND ...</code>
<code>insertar(entidad, objeto)</code>	alta; recupera el id generado	<code>INSERT ... RETURNING id</code>
<code>actualizar(entidad, objeto)</code>	modifica por PK	<code>UPDATE ... WHERE pk = ?</code>
<code>marcar_eliminado(s)(_por)</code>	borrado lógico	<code>UPDATE ... SET eliminado=TRUE WHERE ...</code>
<code>consultar(sql, params)</code>	SQL libre (vistas/JOIN) → lista de dicts	el SQL que se le pase
<code>transaccion()</code>	agrupa varias escrituras en una sola transacción	<code>BEGIN ... COMMIT/ROLLBACK</code>

Compatibilidad de parámetros: los repositorios pasan marcadores estilo `%s`. El gestor los traduce internamente a parámetros nombrados de SQLAlchemy, **siempre parametrizados** (nunca concatena valores → inmune a inyección SQL):

```
# Lo que escribe el repositorio:
listar("postulacion", where="estado_validacion = %s", params=("Pendiente",))
# Lo que ejecuta el gestor (parametrizado):
SELECT ... FROM practicas.postulacion WHERE eliminado = FALSE AND
estado_validacion = :_p0
```

4. Cómo se conecta el programa a la base de datos

4.1 Configuración (`persistencia/config_bd.py`)

Los datos de conexión salen de **variables de entorno** (buena práctica: no incrustar credenciales en el código), con valores por defecto para desarrollo local:

```

CONFIG_BD = {
    "host": os.environ.get("PGHOST", "localhost"),
    "port": int(os.environ.get("PGPORT", "5432")),
    "dbname": os.environ.get("PGDATABASE", "practicas_db"),
    "user": os.environ.get("PGUSER", "postgres"),
    "password": os.environ.get("PGPASSWORD", "postgresql"),
    "schema": os.environ.get("PGSCHEMA", "practicas"),
}

```

4.2 El Engine y la sesión

Con esa configuración se crea **una sola vez** un **Engine** (la "fábrica" de conexiones, que además mantiene un *pool* de conexiones reutilizables) y un fabricante de sesiones:

```

_engine = create_engine(
    URL.create("postgresql+psycopg2", username=..., password=..., host=...,
port=..., database=...),
    client_encoding="utf8",
)
_Session = sessionmaker(bind=_engine)

```

- **postgresql+psycopg2** = motor PostgreSQL usando el driver psycopg2.
- La **Session** es el objeto que ejecuta consultas y lleva la cuenta de los cambios pendientes (qué objetos hay que insertar/actualizar) hasta que se confirma.

4.3 Arranque de la aplicación (main.py)

```

app = QApplication(sys.argv)
GestorPersistencia.inicializar_datos_si_vacio()      # (1) crea esquema + datos de
ejemplo
controlador_login = ControladorLogin(GestorPersistencia()) # (2) gestor para la
app
controlador_login.iniciar()                          # (3) muestra el login
sys.exit(app.exec())

```

En (1), al construir un **GestorPersistencia** se ejecuta **_asegurar_esquema()**, que es **idempotente**: crea el esquema, las tablas, los índices, las restricciones y las vistas **solo si no existen**. Por eso la app funciona contra una base vacía sin pasos manuales:

```

def _asegurar_esquema(self):
    with _engine.begin() as conn:
        conn.execute(text('CREATE SCHEMA IF NOT EXISTS "practicas"'))
        metadata.create_all(_engine, checkfirst=True) # tablas/índices/constraints
    with _engine.begin() as conn:

```

```
for ddl in _ddl_vistas(self.schema):          # CREATE OR REPLACE VIEW ...
    conn.execute(text(ddl))
```

Si además la tabla `login` está vacía, siembra el administrador y un set de datos de ejemplo (`inicializar_datos_si_vacio`).

5. Flujo completo de una CONSULTA mostrada en tabla

Ejemplo real: el coordinador abre "**Validar Postulaciones**" y ve una tabla con las postulaciones pendientes (que cruzan datos de 4 tablas). Seguimos el recorrido:

Paso 1 — La VISTA define la tabla (`ValidarPostulaciones.py`)

```
self.tblPostulaciones = QtWidgets.QTableWidget(parent=Form)
self.tblPostulaciones.setColumnCount(7)
self.tblPostulaciones.setHorizontalHeaderLabels(
    ["ID Postulación", "Cédula Estudiante", "Nombres", "Ciclo",
     "Prácticas Previas", "Empresa/Oferata", "Estado"])
```

Paso 2 — El CONTROLADOR pide los datos (`controlador_coordinador_vinculacion.py`)

```
def refrescar_tabla_validacion(self):
    pendientes = self.repo_postulaciones.detalle_pendientes()      # ← consulta
    self.pintar_tabla(self.v_validar.tblPostulaciones, pendientes,
    self.fila_validacion)
```

Paso 3 — El REPOSITORIO traduce a una consulta sobre la vista (`modelo/proceso.py`)

```
def detalle_pendientes(self):
    s = self.persistencia.schema
    return self.persistencia.consultar(
        f'SELECT * FROM "{s}".vista_postulacion_detalle '
        f'WHERE estado_validacion = %s AND eliminado = FALSE ORDER BY '
        f'id_postulacion',
        ("Pendiente",))
```

Paso 4 — El GESTOR ejecuta el SQL contra PostgreSQL (`gestor_persistencia.py`)

`consultar()` parametriza el `%s`, ejecuta la consulta mediante la sesión/psycopg2, **formatea las fechas** a `dd/MM/yyyy` y devuelve una **lista de diccionarios** `{columna: valor}`:

```
def consultar(self, sql, params=()):
    sql2, binds = self._bindize(sql, tuple(params))    # %s -> :_p0
```



```
filas = self._session.execute(text(sql2).bindparams(**binds)).mappings().all()
# ... convierte DATE -> "dd/MM/yyyy" ...
return [ {col: valor, ...}, ... ]
```

El SQL que finalmente corre en PostgreSQL (aprovechando la vista con sus **JOIN**):

```
SELECT * FROM practicas.vista_postulacion_detalle
WHERE estado_validacion = 'Pendiente' AND eliminado = FALSE
ORDER BY id_postulacion;
```

Paso 5 — El CONTROLADOR vuelca los dicts en el **QTableWidget**

```
def fila_validacion(self, d):                # d = una fila (dict) de la vista
    return [ d["id_postulacion"], d["cedula_estudiante"],
             f'{d["est_nombres"]} {d["est_apellidos"]}', str(d["est_ciclo"]),
             str(d["est_num_practicas"]),
             f'{d["oferta_puesto"]} - {d["nombre_empresa"]}',
             d["estado_validacion"] ]

@staticmethod
def pintar_tabla(tabla, lista, fila_func):
    tabla.setRowCount(len(lista))
    # por cada fila y columna: tabla.setItem(i, j, QTableWidgetItem(str(valor)))
```

Resumen del viaje de una consulta:

```
click → controlador.refrescar() → repo.detalle_pendientes()
      → gestor.consultar(SQL) → [psycopg2 → PostgreSQL → filas]
      → lista de dicts → controlador.pintar_tabla() → QTableWidgetItem en pantalla
```

6. Flujo completo de INSERTAR datos

Ejemplo real: el coordinador registra una empresa (tutor empresarial) en un formulario.

Paso 1 — La VISTA recoge los campos y el CONTROLADOR los procesa

```
def slot_registrar_empresa(self):
    with self.persistencia.transaccion():                # ← todo o nada
        nuevo = self.repo_empresas.agregar(
            self.v_empresas.txtCedula.text().strip(),
            self.v_empresas.txtContraseña.text().strip(),
            ... )                                         # demás campos
        SincronizadorCredenciales.agregar(self.persistencia, nuevo.cedula,
```

```
nuevo.contrasena, TutorEmpresarial.ROL)
self.refrescar_tabla_empresas()
```

Las dos escrituras (la empresa **y** su credencial de login) van dentro de `with persistencia.transaccion():`, así que **se confirman juntas o no se confirma ninguna** (atomicidad).

Paso 2 — El REPOSITORIO valida y crea el objeto del dominio (`modelo/coordinadores.py`)

```
def agregar(self, cedula, contrasena, ...):
    # ... valida campos ...
    nuevo = TutorEmpresarial(cedula, contrasena, ...) # el __init__ valida
    reglas
    self.persistencia.insertar(self.ENTIDAD, nuevo)
    return nuevo
```

Paso 3 — El GESTOR ejecuta el **INSERT** (con SQLAlchemy)

```
def insertar(self, entidad, objeto):
    tabla, clase, _pk = self._entidad(entidad)
    self._session.add(objeto) # marca el objeto como "por insertar"
    self._session.flush() # ejecuta el INSERT (recupera ids IDENTITY)
    self._commit_si_corresponde() # confirma (salvo dentro de transaccion())
```

SQL ejecutado (la contraseña ya viaja cifrada gracias a `ContrasenaHash`):

```
INSERT INTO practicas.tutor_empresarial
(cedula, contrasena, nombres, ..., ruc_empresa, ...)
VALUES ('0107778889', 'pbkdf2_sha256$...', 'Roberto', ..., '0101010106001', ...);
```

Para entidades con id generado (oferta, postulación, ...), el **INSERT** lleva **RETURNING id_...** y el gestor **asigna el id de vuelta al objeto**, para que el resto del flujo ya lo conozca:

```
INSERT INTO practicas.oferta (descripcion, puesto, ...) VALUES (...) RETURNING
id_oferta;
```

7. Actualizar y eliminar (borrado lógico en cascada)

Actualizar (ej. aprobar una postulación): el controlador modifica el objeto y llama `actualizar`; el gestor usa `session.merge` → `UPDATE ... WHERE pk = ?`.

```

postulacion.estado_validacion = "Validada"
postulacion.id_coordinador = self.cedula_coordinador
self.repo_postulaciones.actualizar(postulacion)      # UPDATE practicas.postulacion
SET ...

```

Eliminar (ej. dar de baja una empresa): es **lógico** y **en cascada**, dentro de una transacción para que sea atómico:

```

with self.persistencia.transaccion():
    self.repo_empresas.eliminar(cedula)                # eliminado = TRUE
    SincronizadorCredenciales.eliminar(self.persistencia, cedula)
    eliminacion_cascada.por_empresa(self.persistencia, cedula,
    empresa.ruc_empresa)

```

`eliminacion_cascada.por_empresa` recorre ofertas → postulaciones → prácticas → formularios de esa empresa y los marca `eliminado = TRUE` en lote:

```

UPDATE practicas.oferta      SET eliminado = TRUE WHERE id_oferta      =
ANY(:ids);
UPDATE practicas.postulacion SET eliminado = TRUE WHERE id_postulacion =
ANY(:ids);
UPDATE practicas.practica    SET eliminado = TRUE WHERE id_practica    =
ANY(:ids);
-- formulario1/2/3 igual, por id_practica

```

Como todo está en `with transaccion()`, si algo falla a la mitad **se revierte completo** (no quedan bajas a medias).

8. Transacciones y atomicidad

```

@contextmanager
def transaccion(self):
    # marca que estamos en transacción; al salir bien -> commit; si hay excepción
    -> rollback
    ...

```

- **Fuera** de `transaccion()`: cada escritura confirma sola (autocommit por operación).
- **Dentro** de `with gestor.transaccion()`: todas las escrituras del bloque forman **una sola transacción** → o se confirman todas (**COMMIT**) o se revierten todas (**ROLLBACK**).
- Soporta **anidamiento**: solo el bloque más externo confirma.

Esto garantiza operaciones de negocio consistentes: "registrar empresa + su credencial", "aceptar estudiante + crear práctica", "baja en cascada", etc.

9. Seguridad (puntos fuertes para la defensa)

1. **Sin inyección SQL:** todos los valores van como **parámetros** (`:_p0,%s`), nunca concatenados. Los únicos textos interpolados son nombres de tabla/esquema internos (constantes del programa), no entradas del usuario.
 2. **Contraseñas cifradas:** PBKDF2-HMAC-SHA256 con sal por contraseña; nunca texto plano. El cifrado es automático al escribir (`ContrasenaHash`).
 3. **Integridad garantizada por la base:** FK + `CHECK` + `UNIQUE` + `NOT NULL`. Aunque la app tuviera un bug, la base **rechaza** datos inválidos.
 4. **Credenciales fuera del código:** se leen de variables de entorno.
-

10. Guion rápido de exposición (resumen)

1. **Arquitectura:** Vista (PyQt6) → Controlador → Repositorio/Modelo → Persistencia → PostgreSQL. El SQL está concentrado en `GestorPersistencia`.
2. **Esquema** (`esquema_postgresql.sql`): 13 tablas en el esquema `practicas`, con claves naturales y subrogadas (`IDENTITY`), FK, `CHECK/UNIQUE/NOT NULL`, tipos `DATE/NUMERIC/JSONB`, borrado lógico y vistas para los `JOIN`. La app lo recrea sola al arrancar.
3. **Persistencia con SQLAlchemy:** declara las mismas tablas como objetos `Table`, las **mapea** a las clases del dominio (mapeo imperativo, sin tocar `modelo/`), y convierte tipos con `TypeDecorator` (fecha, hash, `JSONB`).
4. **Conexión:** `config_bd` → `Engine` (pool) + `Session` → `psycopg2` → PostgreSQL; el esquema se crea de forma idempotente en el arranque.
5. **Consultar y mostrar:** controlador pide → `consultar/listar` ejecuta el `SELECT` (usando vistas para los `JOIN`) → devuelve objetos/dicts → se vuelcan en un `QTableWidget`.
6. **Insertar/actualizar/eliminar:** el controlador llama al repositorio → el gestor hace `INSERT ... RETURNING` / `UPDATE` → todo dentro de `transaccion()` cuando debe ser atómico; las bajas son lógicas y en cascada.

Posibles preguntas del tribunal

- ¿Por qué un ORM y no SQL puro? Menos código repetitivo, materialización automática de objetos, parametrización segura y portabilidad, manteniendo el control del esquema.
- ¿Por qué borrado lógico? Conserva historial y nunca rompe integridad referencial.
- ¿Cómo evitan inyección SQL? Parámetros enlazados en todas las consultas.
- ¿Dónde están los `JOIN`? En vistas SQL, reutilizables y eficientes en la base.
- ¿Cómo se genera el id de una oferta/postulación? La base con `IDENTITY`; la app lo recupera con `INSERT ... RETURNING`.